TACTIQ

Product and System Guide

How the Tactiq backend works: what it stores, how data moves through it, how the parts fit together, and exactly what users get on the Free and Pro plans.

Backend version 1 · Written 12 November 2026 · Describes the code in the `Tactiq` repository (Express API on Netlify, jobs on GitHub Actions).
Working product name "Tactiq" is a placeholder (validate naming and trademarks before launch).

Contents

1 Tactiq in one page

2 System architecture

3 Data flow

4 Data we store

5 What the app works out from the data

6 Pricing tiers: Free and Pro

7 Subscriptions and billing

8 Ads

9 Security and privacy

10 Running it: schedule, costs and limits

11 What is built and what is next

How to read this guide

Sections 1–3 explain the system as a whole. Section 4 lists every piece of data the backend keeps, where it lives and why. Section 5 describes the football analysis. Sections 6–8 cover money: plans, subscriptions and ads. Sections 9–11 cover safety, operations and what remains.

A few terms are used throughout:

Term	Meaning
Event data	A log of every on-ball action in a match (pass, shot, tackle, …) with the time, player and pitch position. About 3,000–4,500 events per match.
Freeze frame	The positions of the visible players at the moment of a shot or other key event (StatsBomb "360" data).
Tracking data	Continuous positions of all 22 players many times a second. Tactiq has none yet, so spatial team-shape features are marked unavailable.
Metric	A number computed for a team or player in a match, e.g. PPDA (how aggressively a team presses).
Insight	A plain-language card ("Argentina kept finding players just outside the box") built from metrics, with its evidence.
Entitlement	What a user's plan allows for one feature, e.g. "20 analysed matches a month".
Job	A piece of background work (download a match, analyse it, …) recorded as a row in the database.

1 · Tactiq in one page

Tactiq is a fan-first football tactics app for Android. It explains *why* a match went the way it did, in plain language first, with the evidence and deeper numbers one tap away.

Who it is for

- **Casual fans**: the match story and simple explanations.
- **Engaged fans**: pressing, build-up, player roles, comparisons.
- **Tactical enthusiasts**: metric definitions, filters, evidence.
- **Coaches, scouts, analysts**: the same data at Advanced depth.

These are not separate accounts. Everyone has one account and picks a depth (Quick, Explore, Advanced) whenever they like.

What the backend does

- Downloads match data from a provider (today: StatsBomb Open Data).
- Cleans it into one consistent format and stores it.
- Computes ~45 tactical metrics per match, then insight cards, a match story and team playing styles.
- Serves it all to the app through a versioned REST API.
- Keeps each user's follows, saves, notes and collections private.
- Runs the Free/Pro paywall, Google Play subscriptions and ad rules.

The main idea behind the design

> **Analysis is done once per match, in the background, and stored.** When a user opens a match, the API only reads finished results from the database. No user ever waits for a calculation, and the API can run on cheap serverless hosting.

The product in numbers (backend v1)

Area	Count	Notes
API routes	~69	All under `/v1` ; full contract in `docs/API.md`
Database tables	43	Shared football data, analysis, user data, billing, jobs
Metric definitions	47	44 active, 3 waiting for tracking data
Insight rules	11	Turn metrics into plain-language cards
Player role archetypes	14	From goalkeeper to poacher
Glossary concepts	31	Each with simple, explore and advanced text
Plans	2	Free, and Pro (sold monthly or annually)
Automated tests	65	Including end-to-end tests of the paywall and billing

2 · System architecture

Tactiq is a small set of managed services around one Express application. The same code runs in two places: as the **API** (on Netlify) answering the app, and as the **worker** (on GitHub Actions) doing background work. They never talk to each other directly; they share the database.

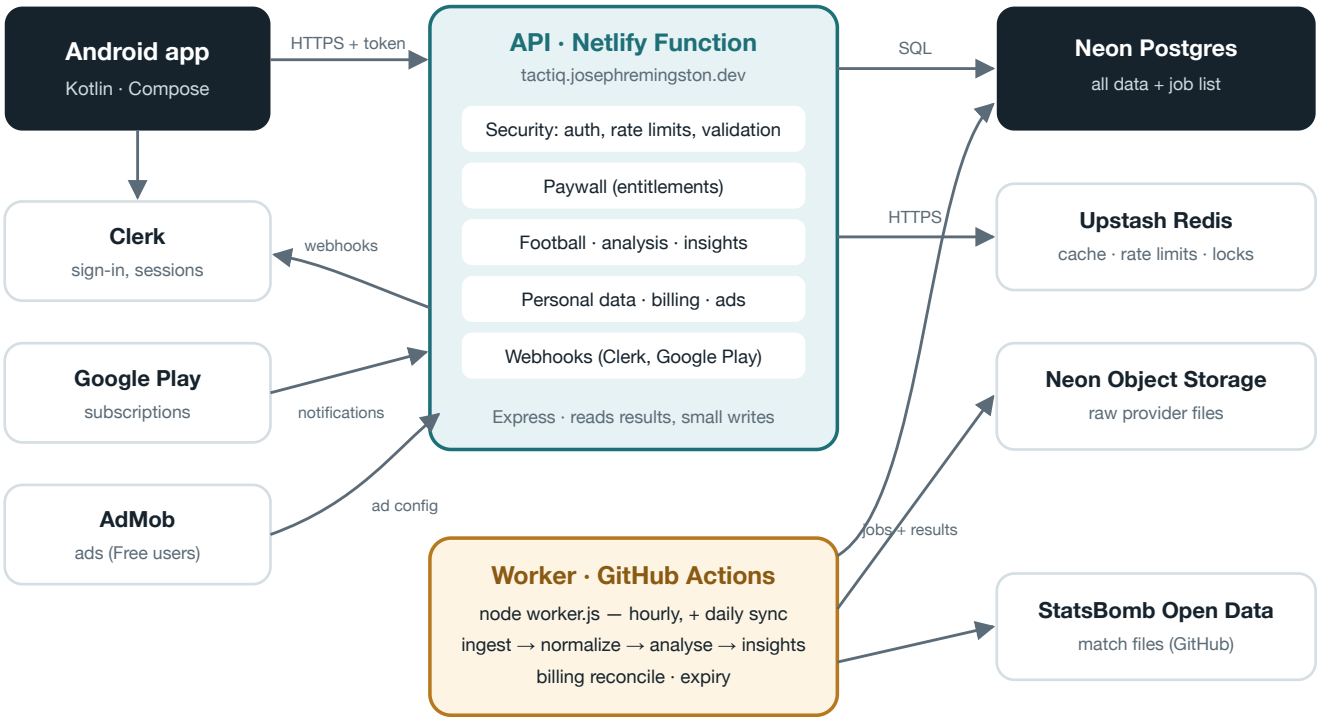


Figure 1. Components. Teal: the API answering the app. Amber: the background worker. Both run the same codebase.

What each part does

Component	Role	Why this choice
API (Express on Netlify Functions)	Answers every app request. Checks the user, the paywall and rate limits, then reads or writes the database.	Real Node.js, so Express, Postgres and Clerk work without workarounds; your domain's DNS is already on Netlify, so the custom subdomain attaches directly. Free tier.
Worker (`node worker.js` on GitHub Actions)	Runs background jobs: downloading matches, normalising them, analysing them, billing checks.	Analysis needs seconds of CPU and up to 9 MB files; serverless request limits are too tight. Actions gives free minutes and a scheduler.
Neon Postgres	The source of truth: football data, analysis results, users, plans, purchases, and the job list itself.	Managed Postgres with a pooled connection string suited to serverless; transactions keep billing and usage counting correct.
Neon Object Storage	Keeps the original provider files (raw JSON) exactly as downloaded.	Large files do not belong in database rows; keeping originals allows re-processing and audits (licence permitting).

Component	Role	Why this choice
Upstash Redis	Short-lived cache, rate-limit counters and locks. Optional: if it is down, the app still works, just uncached and unthrottled.	Reached over HTTPS, so it works from serverless functions without TCP connections.
Clerk	User sign-in (Google, passkeys, email) and sessions.	No passwords or OTP flows to build or secure ourselves. Tokens are verified locally with a public key, without a network call.
Google Play Billing	Takes payment for Pro. The backend verifies every purchase with Google.	Required for digital subscriptions sold in a Play Store app.
AdMob	Shows ads to Free users in low-interruption places.	Recommended in the product plan; the backend controls where ads may appear.
StatsBomb Open Data	Today's source of matches, lineups, events and freeze frames.	Free and rich, ideal for building and validating. Not licensed for commercial redistribution (see section 9).

Inside the codebase

The repository follows the same layout as the HireMind project. Each request passes through layers in this order:

Layer	Folder	Responsibility
Routes	`routes/`	Maps a URL to its middleware and controller.
Middlewares	`middlewares/`	Sign-in check, input validation, rate limits, paywall gates.
Controllers	`controllers/`	Read the validated request, call a service, send the standard response.
Services	`services/`	All business logic: football queries, analysis engine, insights, billing, ads.
Models	`models/`	Database table definitions.
Queues / Workers	`queues/` , `workers/`	Create job rows / run them.
Configs / Utils	`configs/` , `utils/`	Connections to Postgres, Redis, storage, Clerk, Google Play; shared helpers.
Data	`data/`	Plans, ad placements, metric definitions, glossary, xT grid, role archetypes.

3 · Data flow

Four flows cover everything the system does: getting football data in, a user reading it, a user signing in, and a user paying.

3.1 Getting football data in (background)

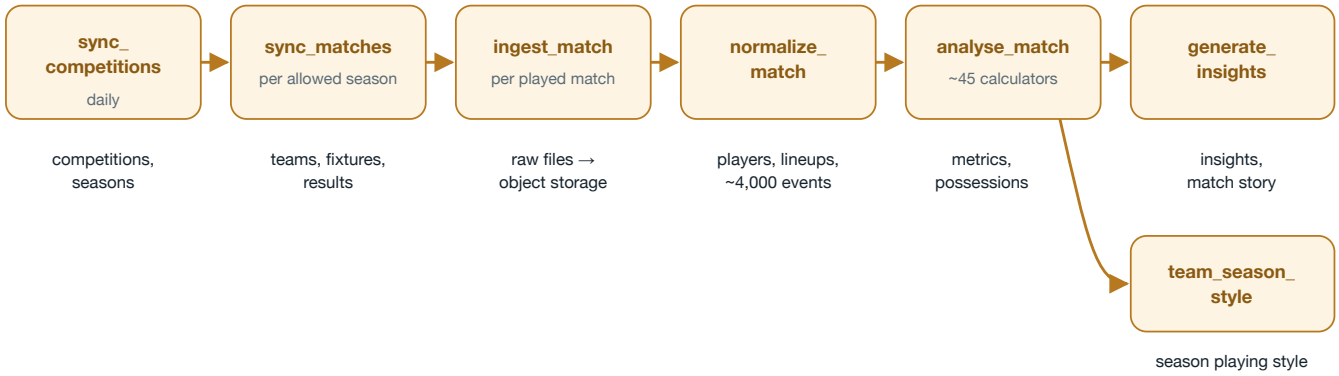


Figure 2. The job chain. Each job queues the next. Below each job is what it writes.

1. **sync_competitions** downloads the competitions list and stores competitions and seasons. It only continues for seasons on the allowlist (today: FIFA World Cup 2022 and 2018).
2. **sync_matches** stores teams, fixtures and results for each allowed season, and queues one ingest job per played match.
3. **ingest_match** downloads the lineups, events (2–5 MB) and freeze frames (up to 9 MB) and saves the original files in object storage. If the files have not changed since last time, it stops here.
4. **normalize_match** converts the provider's format into Tactiq's own: one row per player, lineup entry and event, with pitch coordinates converted to metres and every team attacking left to right. It also records a data-quality report.
5. **analyse_match** runs every calculator the match's data supports and stores the metrics and possession chains in one atomic step.
6. **generate_insights** turns those metrics into ranked insight cards and the match story.
7. **team_season_style** updates each team's playing-style profile for the season.

> **Safe to repeat.** Every job has an id derived from its inputs, so queueing the same work twice does nothing. Rows are written with "insert or update", so re-running the whole chain never creates duplicates. Failed jobs retry with growing delays (up to 5 tries) before being marked *dead* for a person to replay.

3.2 A user opening a match (live request)

Step	What happens
1. Request	The app calls e.g. `GET /v1/matches/{id}/story` with the user's Clerk token.
2. Security	Security headers are set, the IP is rate-limited, the token is verified with Clerk's public key (no network call), and the local user is loaded (cached for 5 minutes).
3. Validation	Parameters are checked strictly; anything malformed gets a 400 before touching the database.

Step	What happens
4. Paywall	For deeper views only: the entitlement is checked and one unit is counted (once per match per month).
5. Read	The service reads the stored results from Postgres, often through the Redis cache.
6. Response	The standard envelope `{ statusCode, message, data }` is returned, with any ad slots for Free users.

3.3 Signing in

1. The app signs the user in with Clerk (Google, passkey or email). Tactiq never sees a password.
2. Clerk gives the app a short-lived session token, sent on every API call.
3. On the user's first call, the backend creates their Tactiq account, copies their name, email and avatar from Clerk, and gives them the Free plan.
4. Clerk also sends signed webhooks when a user changes their profile or is deleted, so Tactiq stays in sync.

3.4 Paying for Pro

1. The app shows the plan comparison from the API and the price from Google Play.
2. The user buys through Google Play. The purchase carries a hashed Tactiq account id, so it cannot be claimed by another account.
3. The app sends the purchase token to the backend, which asks Google directly whether it is real and active.
4. If it is, the backend records the purchase, grants the Pro plan until the paid-through date, and acknowledges the purchase to Google (unacknowledged purchases are refunded by Google after 3 days).
5. From then on, Google notifies the backend of every change (renewal, payment problem, cancellation, refund). For each one the backend re-reads the full state from Google and adjusts access.
6. A daily job re-checks purchases in case a notification was lost.

3.5 Background job lifecycle

Status	Meaning
queued	Waiting for the next worker run (or for its scheduled time, for delayed retries).
active	A worker has claimed it. If that worker dies, the job returns to queued after 30 minutes.
completed / skipped	Done; *skipped* means there was nothing to do (e.g. another sync held the lock).
failed	An attempt failed; it will retry after a delay (1 min, 2 min, 4 min … up to 30 min).
dead	Retries used up, or the provider's data format changed. Needs a person: `node cli.js jobs:replay`.

Only one user action creates background work: **"re-run analysis"** on a match. Everything else a user does is answered immediately.

4 · Data we store

Data falls into three kinds. Shared football data belongs to no one: every user sees the same clubs, matches and analysis. Private data belongs to one user and is only ever returned to that user. Billing data records money and access and is kept for audit.

4.1 Where data lives

Store	What is kept there	Kept for
Neon Postgres	Everything structured: 43 tables listed below.	Permanently, except what account deletion removes.
Neon Object Storage	Original provider files: competitions, season fixtures, lineups, events, freeze frames. One file per version, named by content hash.	Permanently today. A retention rule should follow each provider's licence.
Upstash Redis	Temporary copies and counters only (see 4.7). Nothing that cannot be rebuilt.	Seconds to 6 hours.
Clerk	The user's sign-in identity: email, name, avatar, linked Google account, passkeys, sessions.	Until the user deletes their account.
Google Play	Payment details and the subscription itself. Tactiq never sees card data.	Google's policy.
Logs (Netlify, GitHub Actions)	One JSON line per error or job: request id, path, job id, error message. No tokens or notes.	The host's log retention.

4.2 Shared football data Shared

Table	One row per	Main contents
competitions	competition	Name, country, gender, youth/international flags.
seasons	season of a competition	Name (e.g. "2022"), dates.
teams	club or national team	Name, short name, country, gender.
players	player	Full and known name, birth date, country, main position.
player_team_assignments	player in a team-season	Shirt number, source.
matches	match	Competition, season, teams, kick-off, status, score, stage, stadium, referee, pitch size, analysis status and version.
match_participants	team in a match	Home/away, starting formation, manager.
match_coverage	match	Which data exists (lineups, events, freeze frames, tracking, video), event count, provider, data-quality report, licence.
lineups	player in a match	Position, starter or sub, minutes on/off, cards.

Table	One row per	Main contents
events	on-ball action	Period, clock, type, team, player, possession, coordinates (canonical and original), outcome, pressure flags, type-specific detail (e.g. shot xG).
event_freeze_frames	shot / key event	Visible players' positions at that moment.
tactical_concepts	glossary entry	Simple, explore and advanced explanations, misconceptions, related concepts.
data_licences	data source	Terms summary, attribution text, whether commercial use is allowed.
provider_entity_maps	provider id	Links a provider's id (e.g. StatsBomb team 779) to Tactiq's own id, so providers can be swapped.
raw_payloads	downloaded file	Where the original file is stored, its hash, size, source URL and retrieval time.
provider_sync_state	sync scope	When it was last synced, so unchanged data is skipped.

4.3 Analysis results Shared

Table	One row per	Main contents
metric_definitions	metric version	Name, unit, direction (higher/lower is better), three descriptions, formula, data required, provider caveats.
analysis_runs	analysis of a match	Version, timing, which calculators ran, data available at the time, errors.
tactical_metrics	metric × team/player × scope	Value, details, sample size, minutes, coverage label, evidence (event ids). About 900 per match.
match_possessions	possession chain	Team, start/end, how it ended, passes, whether it reached the final third or box, threat added, xG.
insights	insight card	Headline, what happened, why it matters, evidence, confidence, coverage label, all three depth texts, link to the deeper view.
match_stories	match	Short version, team approaches, key patterns, what changed, key players, coverage statement.
team_season_styles	team-season	Median and spread of 11 style metrics, competition percentiles, plain-language labels.

4.4 Users and their private data Private

Table	Contents	Personal data?
users	Clerk id, verified email, display name, avatar URL, language, preferred depth, interests, status (active/deleted).	Yes: email, name, avatar
user_identities	Linked sign-in identities (e.g. Google account id and email).	Yes
followed_teams / _competitions / _players	What the user follows, and when.	Preferences
saved_matches, saved_insights	Bookmarks, with the time saved.	Preferences
user_notes	Free-text notes attached to a match, team, player or insight (up to 4,000 characters).	Yes: user-written text

Table	Contents	Personal data?
user_collections, _items	Named lists of matches, insights, players, teams or notes.	Preferences
learning_progress	Glossary concepts viewed, saved or completed.	Preferences
security_events	Account creation and deletion, session revocation, purchase checks, rejected webhooks: kind, time, IP, device type, small metadata.	Yes: IP address

4.5 Plans, usage and billing Billing

Table	Contents
plan_definitions	Free, Pro monthly, Pro annual and each plan's feature limits (from `data/plans.json`).
subscriptions	A user's plan period: plan, status, source (Google Play / admin), current period end, cancellation time.
entitlements	What a user may do, one row per feature per grant, with validity dates. Never edited in place: changes add or deactivate rows, so history is auditable.
entitlement_usage	Counters: how many units of a feature a user used in a month.
entitlement_usage_events	Which subject (e.g. which match) was counted, so the same match is never charged twice in a month.
entitlement_audit	Every grant, revoke, expiry and plan change, with who or what did it.
purchases	Google Play purchase: product, purchase token, order id, state, renewal flag, acknowledged flag, start/expiry, last check, Google's last response.
billing_notifications	Each Google Play notification received: message id, type, token, processing status. Prevents processing one twice.

4.6 Background work

Table	Contents
ingestion_jobs	The job list: type, status, attempts, last error, progress, input, result, who requested it (if a user), priority, earliest run time, lock time, trace id linking a whole chain.

4.7 Temporary data in Redis

Key	Holds	Lifetime
`user:clerk:{id}`	The local user behind a Clerk id	5 minutes
`ent:{userId}`	The user's active entitlement grants	60 seconds
`cache:match:{id}:summary`	Match centre header	60 seconds
`cache:match:{id}:run:…`	Analysis views for one analysis run	6 hours
`rl:*`	Rate-limit counters per IP or user	1 minute windows

Key	Holds	Lifetime
`lock:*`	Prevents two provider syncs at once	10 minutes

4.8 What happens when a user deletes their account

Deleted

- Follows, saved matches and insights
- Notes and collections
- Learning progress
- Linked identities
- Usage counters
- Email, name, avatar and interests on the user row
- The Clerk account itself
- The link between the user and their past jobs

Kept

- An anonymous user row marked *deleted* (so ids in logs stay meaningful)
- Purchases, subscriptions and the entitlement audit (financial records, needed for disputes and tax)
- Security events (including the deletion itself)
- All shared football data

5 · What the app works out from the data

Every number is **deterministic**: the same match always gives the same results, each metric has a written definition and a version, and each result carries its evidence (the event ids it was counted from). Each result is labelled **observed** (counted directly), **calculated** (a formula over events) or **inferred** (a model or interpretation), so fans can tell facts from opinions.

5.1 Metrics by family

Family	Metrics
Possession	Possession share, number of possession chains, average passes per chain, final-third entries, box entries, expected threat (xT) added, xT by lane.
Passing	Passes attempted, pass completion, passing network, key passes, expected assists (xA).
Progression	Progressive passes and carries, progressive distance, progression by lane (left/centre/right), receptions in zone 14.
Build-up	Build-up success rate (chains from the back that reach the final third).
Pressing	PPDA, PPDA including pressures, pressures, ball regains within 5 s of a pressure, high turnovers, shots after high turnovers, average height of defensive actions.
Transition	Counter-press regains within 3/5/8 s, counter-press regain rate, counter-attack shots, time from regain to shot.
Shooting	Shots, on target, goals, xG, non-penalty xG, xG per shot, goals minus xG, set-piece share of xG, shot map, penalty shoot-out.
Shape	Starting formation, formation changes over time, average positions. *Planned, need tracking:* team width, compactness, defensive line height.
Player	Minutes, role profile (closest of 14 archetypes, see 5.3).

Each metric is produced for the full match and, where it makes sense, for the first half, second half, extra time, and while leading, drawing or trailing.

5.2 Insights and the match story

Eleven rules look for notable patterns and write them up in plain language at three depths. Each is ranked by how notable it is, how complete the data is and how big the sample is.

Insight	Looks for
High-press dominance	One team pressing far more aggressively than the other (PPDA).
High-turnover threat	Winning the ball high up the pitch and turning it into shots.
Flank dominance	Progressing mostly down one side.
Central access	Repeatedly finding players in zone 14, just outside the box.
Control without threat	Lots of the ball but little danger.
Finishing variance	Scoring far more or less than the chances deserved (goals vs xG).
Second-half shift	A big change in threat or pressing after half-time.

Insight	Looks for
Score-state response	How a team changed when it went ahead or behind.
Formation change	A switch of shape during the match.
Progression hub	The player who moved the ball forward most.
Chance creator	The player who set up the most shot value.

The **match story** combines them: a one-line summary without jargon, how each team tried to play, the three most important patterns (from different families, both teams represented), what changed (goals, substitutions, shifts), the players who shaped the game, and a coverage statement saying what the data could and could not show. When data is thin, the story falls back to a basic recap and says so.

5.3 Player roles

For each player with at least 30 minutes, 17 per-90 features (passes, progressive actions, pressures, box touches, average position, …) are compared with a reference population and matched to the closest of 14 roles within their position family:

Goalkeeper · Ball-playing centre-back · Stopper centre-back · Overlapping full-back · Inverted full-back · Holding midfielder · Deep-lying playmaker · Box-to-box midfielder · Ball-winner · Advanced playmaker · Winger · Target forward · Pressing forward · Poacher

5.4 Team playing style

After at least 5 analysed matches in a season, a team gets plain-language labels such as *presses high*, *builds patiently*, *plays directly*, *dominates the ball*, *attacks quickly after regaining the ball* or *creates high-quality chances*, each backed by medians, spreads and competition percentiles. A label is only given when the team is consistent; varied teams get none rather than a misleading one.

5.5 Glossary

31 tactical concepts, each with a simple sentence, an explore paragraph, an advanced definition, common misconceptions and links to related concepts and metrics: possession, possession chain, build-up, progressive pass, line-breaking pass, half-space, wide channel, zone 14, final third, box entry, overload, high press, mid-block, low block, PPDA, counter-press, rest defence, transition, xG, xA, xT, formation, inverted full-back, false nine, double pivot, third-man run, passing network, set piece, compactness, player role, score state.

6 · Pricing tiers: Free and Pro

There are two plans. Free must give genuine tactical understanding, not just scores. Pro sells depth, volume, history and an ad-free experience. A fan, an enthusiast or a coach should all understand the difference without learning coaching jargon.

Free

For every fan. No card needed.

- Follow any clubs, competitions and players
- Personal home feed
- Full match centre: results, lineups, the complete match timeline
- Every match story and insight card, at all three depths
- The whole tactical glossary and learning progress
- Team playing styles
- **20 matches a month** of detailed analysis
- **5 comparisons** a month
- **3 re-analysis requests** a month
- Latest **2 seasons** of each competition for analysis
- Up to **100 saved** matches and insights
- Unlimited notes and collections
- Light ads in browsing screens only

Pro

Monthly or annual. Same features on both.

- **Everything in Free**, plus:
- **Unlimited** detailed match analysis
- **Unlimited** comparisons
- **30 re-analysis requests** a month
- **Advanced filters** on the timeline and possessions
- **Full history**: every season ever ingested
- **Unlimited** saved matches and insights
- **No ads** anywhere
- Reserved for coming features: AI explainer credits (100/month vs 5), report exports (10/month vs 0), tracking data access

6.1 Feature-by-feature detail

Feature	Free	Pro	What it means for the user
Account, following, feed	**Yes**	**Yes**	Follow any number of clubs, competitions and players; the home feed shows their recent and upcoming matches and top insights.
Search and browse	**Yes**	**Yes**	Every competition, club, player, match and glossary concept.
Match centre	**Yes**	**Yes**	Score, lineups, cards, substitutions, coverage label, licence attribution.
Match timeline	**Full**	**Full + filters**	Free users scroll every event. Filtering by event type, player, team, period or possession, and the raw provider data, are Pro.
Match story and insight cards	**Always**	**Always**	Never paywalled. The plan says a fan should always be able to understand a match without paying.

Feature	Free	Pro	What it means for the user
Glossary and learning	**Full**	**Full**	All 31 concepts and progress tracking.
Team playing style	**Yes**	**Yes**	Season labels and style metrics.
Detailed match analysis	20 matches / month	**Unlimited**	The Explore and Advanced views: all metrics, tactical shape and passing networks, pressing, transitions, possession chains, per-player role analysis. One match counts once a month however many views are opened.
Comparisons	5 / month	**Unlimited**	Side-by-side comparison of two matches (optionally for one team). Each distinct pair counts once a month.
Re-run analysis	3 / month	30 / month	Ask for a match to be analysed again. Only counted when new work is actually queued; already-analysed matches cost nothing.
Advanced filters	**No**	**Yes**	Filtered timeline and possession views.
Historical seasons	Latest 2 per competition	**All**	Detailed analysis of older seasons. Stories and timelines of old matches stay free.
Saved items	100	**Unlimited**	Saved matches and insights combined. Notes and collections are not limited.
Ads	Light	**None**	See section 8.

6.2 How limits are counted

- **Monthly limits reset at midnight UTC on the 1st** of each month. The app shows how many are left and when they reset.
- **One match = one unit per month.** Opening the metrics, then pressing, then a player view of the same match uses one unit, not three.
- **Nothing is charged for a failure.** If the server errors after counting, the unit is refunded.
- **Limits are enforced on the server.** Changing the app cannot unlock anything.
- When a limit is reached the user sees the paywall with the reset date and the plans that remove the limit. The match story and timeline stay available.

6.3 Example month for a Free user

Action	Uses	Left
Reads 15 match stories	nothing	20 matches
Opens pressing + passing network of the derby	1 match	19
Comes back to the derby's player view next week	nothing (same match, same month)	19
Compares the derby with last season's	1 comparison	4 comparisons
Filters the timeline to shots only	paywall (Pro feature)	—
Opens analysis of a 2015 match	paywall (older than latest 2 seasons)	—

6.4 Products and prices

Plan key	Google Play product	Billing period	Price
free	—	—	Free, ad-supported
pro_monthly	`pro_monthly`	1 month	Set in Play Console (not decided yet)
pro_annual	`pro_annual`	1 year	Set in Play Console (not decided yet)

Prices live in Google Play, per country, and the app reads them from Play. The backend never stores a price. The limits above are product hypotheses from the plan; they should be checked against real usage and data costs before launch.

> **Changing what is free.** Every limit lives in one file, `data/plans.json`. To move a feature between Free and Pro or change a number, edit it and run `node cli.js db:seed`. No code changes, no app release. The paywall comparison table shown in the app is in the same file.

7 · Subscriptions and billing

What a Pro user experiences in each state of their Google Play subscription:

State	When it happens	Pro access?	What the app should say
pending	Payment not completed yet (e.g. cash payment)	**No**	"We'll unlock Pro when your payment completes."
active	Paid and renewing	**Yes**	Renewal date.
grace	Renewal payment failed; Google is retrying	**Yes**	"Update your payment method to keep Pro."
on_hold	Grace period ran out	**No**	"Pro is paused until payment is fixed."
paused	User paused the subscription	**No**	Resume date.
cancelled	User cancelled; period not over	**Until period end**	"Pro until {date}."
expired	Period over, not renewed	**No**	Offer to resubscribe.
revoked	Refunded or charged back	**No**	—

- **Nothing is lost when Pro ends.** Saved items, notes and collections stay; only new saves beyond 100 and Pro views are locked.
- **Upgrading or switching** monthly ↔ annual: Google issues a new purchase that replaces the old one; the backend ends the old grant and starts the new one.
- **Restore purchases** re-reads every purchase the user has made, for new phones and reinstalls.
- **Fraud protection:** every purchase is checked with Google; a purchase token can belong to one account only; purchases carry a hashed account id; notifications are authenticated and processed once.
- **Manual grants:** support can grant Pro for a period with `node cli.js entitlements:grant`, e.g. for testers or compensation.

> **Not live yet.** Billing switches on once the Android app is in Play Console and three settings are added (package name, service-account key, notification token). Until then the purchase endpoints answer "billing not configured".

8 · Ads

Ads are shown only to Free users, through Google AdMob, in a few low-interruption places. The backend decides whether and where; the app only displays them. Placements can therefore change without an app update.

Placement	Screen	Format	Position
home_feed_native	Home	Native card	After the 4th recent match, then every 6; max 3 per session
explore_feed_native	Explore / search / match lists	Native card	After the 5th result, then every 8; max 3 per session
match_list_banner	Match lists	Small banner	Bottom of screen

Never shown on: sign-in, onboarding, subscription and payment, the match story, match centre and match review, the tactical pitch, advanced analysis, reports, privacy, account deletion and settings. Overall at most 6 ads per session, at least 60 seconds apart. No rewarded ads: a fan never watches an ad to understand a match.

Privacy: the ad configuration contains placement data only. No player data, notes, follows or anything about the user is sent towards ad networks. Consent is collected in the app with Google's consent SDK. Outside production, Google's test ad units are used automatically.

9 · Security and privacy

9.1 Protections in place

Threat	Protection
Someone reading another user's data	The user id always comes from the verified sign-in token, never from the request. Every private query is filtered by owner. Another user's item answers "not found", hiding that it exists. Tested automatically.
Fake or stolen sign-in tokens	Tokens are checked against Clerk's public key, expiry and allowed origins on every request.
Unlocking Pro by tampering with the app	Every limit is enforced on the server; purchases are verified with Google.
Abuse and scraping	Rate limits per IP (120/min) and per user (600/min), tighter on job creation (10/min) and purchase checks (20/min). Client-IP headers are trusted only when set by Netlify or a verified Cloudflare edge.
Malformed or oversized input	Strict validation of every parameter and body; unknown fields rejected; 256 KB body limit.
Forged webhooks	Clerk webhooks are signature-checked; Google notifications need a secret token; both are processed at most once.
Leaking internals	Errors never include stack traces or SQL; each has a request id for support.
Secret exposure	Secrets live only in Netlify and GitHub's secret stores; the git history was checked and contains none.
Network attacks	HTTPS everywhere, security headers (HSTS, no-sniff, frame protection). An optional switch lets Cloudflare front the API later and block direct access.

9.2 Before launch

- **Data licence.** StatsBomb Open Data is for development; it is not licensed for a commercial customer-facing app. A licensed provider (e.g. Sportmonks) must be contracted before launch. Every match screen shows the data source.
- **Privacy policy** and terms describing the data in section 4, and a review of India's DPDP Act and other markets' rules.
- **Retention** rules for raw provider files and security logs.
- **Penetration test** and a backup restore drill (Neon point-in-time restore).

10 · Running it: schedule, costs and limits

10.1 Schedule

When	What runs
Every request	The API on Netlify.
Every hour (:17)	Worker drains queued jobs: re-analysis requests, retries, daily billing jobs.
Daily (03:40 UTC)	Worker syncs competitions and matches from the provider, then drains.
Daily (first run)	Billing reconciliation and expiry of ended plans.
Every push	65 automated tests on GitHub Actions.

10.2 Costs (today)

Service	Plan	Notes
Netlify	Free	Function invocations and bandwidth within the free allowance at launch scale.
GitHub Actions	Free / Student Pack	~800–1,100 minutes a month for a private repo (3,000 included with GitHub Pro via the Student Pack); unlimited for a public repo.
Neon Postgres + storage	Free	Watch storage as matches grow: ~4,000 events and ~900 metrics per match.
Upstash Redis	Free	Billed per command; caching and rate limits use a few commands per request.
Clerk	Free	Free up to its monthly active user limit.
Google Play	15% / 30% fee	Google's service fee on subscriptions.

10.3 Known limits

- A user's re-analysis request waits for the next hourly run. It can be made near-instant by letting the API trigger the worker directly.
- No tracking or video data yet, so team width, compactness, defensive line, off-ball runs and physical load are shown as unavailable.
- Team style labels need 5+ analysed matches per team-season; with only two World Cup seasons ingested most teams have none yet.
- Each Netlify request has a time limit; all endpoints are simple reads, but very large pages (500 events) are the heaviest.

10.4 Everyday commands

Command	Does
`npm run dev`	Runs the API locally.
`node worker.js [--sync]`	Drains the job queue once (optionally syncing first).

Command	Does
`node cli.js db:migrate · db:seed`	Applies database changes · loads plans, metrics and glossary.
`node cli.js sync:statsbomb --inline`	Syncs and analyses data in one go.
`node cli.js jobs:replay --dead-since 24`	Retries every job that died in the last day.
`node cli.js entitlements:grant --user … --plan pro_monthly`	Gives a user Pro by hand.

11 · What is built and what is next

11.1 Built and tested

- Discovery and match centre for every ingested competition, with honest coverage labels.
- Full analytics pipeline, metrics, insights, match stories, player roles and team styles.
- Accounts, following, feed, saves, notes, collections and learning progress, with ownership checks.
- Free/Pro paywall, Google Play subscription verification and notifications, server-driven ads.
- Security: sign-in, rate limits, validation, signed webhooks, audit log.
- Production database migrated and seeded.

11.2 To go live

1. Commit and push the restructured repository.
2. Set Netlify environment variables and attach `tactiq.josephremingston.dev`.
3. Add GitHub Actions secrets and run the first data sync.
4. Set the real Clerk allowed origins and webhook secret.
5. When the Android app is in Play Console: Google Play billing settings, AdMob ad units, test purchases.
6. Before a commercial launch: a licensed data provider, privacy policy, security review.

11.3 Later milestones (from the product plan)

Milestone	Adds
4 · Tracking	Licensed tracking data; team shape, compactness, line heights, off-ball runs, space occupation.
5 · Reports and video	PDF reports, video clips synced to events (where rights allow), push notifications.
7 · AI explainer	"Ask about this match": answers grounded in stored metrics and events, using the reserved AI credits.
7 · Pattern discovery	Automatically found repeated sequences, opponent reports, personal tactical models.
8 · Launch and scale	Security audit, load tests, provider sign-off, dashboards and runbooks.